

PROJECT REPORT

Hospital Virtual Appointment Booking Queue System

Submitted To: Department of Computer Science and Information Technology, Pokhara University

Submitted By: Student Name: Aayushma Joshi | Roll No.: 24080602

College: Rajdhani Model College

Degree: Bachelor of Computer System and Information Technology (BCSIT)

Academic Year: 2026/20

Certificate

This is to certify that the project entitled "**Hospital Virtual Appointment Booking Queue System**" submitted by **Aayushma Joshi** in partial fulfillment of the requirements for the degree of **Bachelor of Computer System and Information Technology (BCSIT)**, Pokhara University, is an original work carried out under my supervision.

Supervisor's Signature: _____

Date: _____

Acknowledgement

I would like to express my sincere gratitude to my project supervisor for the continuous guidance, valuable suggestions, and unwavering encouragement provided throughout the development of the Hospital Virtual Appointment Booking Queue System. The expertise and constructive feedback I received played a significant role in the successful completion of this project.

I am also deeply thankful to the faculty members of the Department of Computer Science and Information Technology at Rajdhani Model College for imparting the foundational knowledge and technical skills necessary to undertake a software development project of this scale.

Furthermore, I extend my appreciation to my friends and family for their endless moral support and motivation during the challenging phases of system design and implementation. This project would not have been possible without their collective support.

Abstract

The **Hospital Virtual Appointment Booking Queue System** is a comprehensive, web-based application engineered to digitalize and streamline the appointment scheduling and queue management processes within healthcare facilities. Traditionally, hospitals rely on manual or disjointed digital systems to manage patient inflow, which frequently results in overcrowded waiting rooms, unpredictable waiting times, and administrative bottlenecks. This project addresses these critical inefficiencies by introducing an automated platform that manages the entire lifecycle of a patient's outpatient visit.

Developed utilizing a robust technology stack—including **Java 21**, **Spring Boot 3.2.5** for backend logic, **Spring Security 6** for role-based access control, **Spring Data JPA with MySQL** for persistent data storage, **Google Gmail API (OAuth 2.0)** for automated email communications, and **Thymeleaf integrated with Tailwind CSS** for a responsive frontend—the system delivers a seamless experience across three distinct user roles: Patient, Doctor, and Administrator.

By implementing algorithmic queue generation and priority scoring (distinguishing between normal and emergency appointments), the system significantly optimizes patient throughput. The integration of modern UI components ensures accessibility across mobile and desktop devices. Ultimately, the Hospital Virtual Appointment Booking Queue System minimizes physical congestion in healthcare facilities, enhances the accuracy of medical administrative records, and substantially improves the overall healthcare experience for both providers and patients.

Table of Contents

1. Chapter 1: Introduction
2. Chapter 2: Problem Statement
3. Chapter 3: Objectives
4. Chapter 4: Scope of the Project
5. Chapter 5: Literature Review
6. Chapter 6: System Analysis
7. Chapter 7: Feasibility Study
8. Chapter 8: Requirement Analysis
9. Chapter 9: System Design & Diagrams
10. Chapter 10: Database Design & ER Diagram
11. Chapter 11: Implementation Details
12. Chapter 12: Testing
13. Chapter 13: Results & Discussion
14. Chapter 14: Conclusion
15. Chapter 15: Future Enhancements
16. Chapter 16: References

Chapter 1: Introduction

The rapid advancement of information technology has fundamentally transformed various sectors, with healthcare being one of the primary beneficiaries. Despite these technological strides, many healthcare institutions continue to struggle with outpatient department (OPD) management. The traditional process of visiting a hospital, waiting in long physical queues to book an appointment, and subsequently waiting outside a doctor's cabin for an indeterminate amount of time is highly inefficient. It induces immense stress on patients and imposes a heavy administrative burden on hospital staff.

The Hospital Virtual Appointment Booking Queue System is a web-based software solution designed to eliminate the friction associated with medical appointments. By shifting the queueing paradigm from a physical space to a virtual platform, the system allows patients to secure their position in a doctor's schedule remotely.

Chapter 2: Problem Statement

The administration of patient flow in modern healthcare facilities is fraught with logistical challenges. Core issues prevalent in traditional operational models include:

- **Unpredictable Waiting Times and Overcrowding:** Patients arrive hours before consultations without visibility.
- **Inefficient Handling of Emergencies:** Walk-in emergency cases disrupt daily schedules without automated priority sorting.
- **High Rate of Administrative Errors:** Manual transcription leads to double-bookings and lost appointment records.
- **Lack of Real-Time Communication:** Patients are not notified when consultations run over time.
- **Absence of Centralized Analytics:** Hospital administrators lack real-time visibility into daily operations.

Chapter 3: Objectives

General Objective

To design, develop, and deploy a computerized Hospital Virtual Appointment Booking Queue System that comprehensively manages the outpatient workflow, seamlessly connecting patients, doctors, and hospital administrators through a centralized virtual platform.

Specific Objectives

- Digitalize Patient Onboarding with 6-digit email OTP verification.
- Streamline Appointment Scheduling with doctor filtering by specialization.
- Automate Queue Management with priority scoring (Emergency = 1, Normal = 0).
- Integrate Google Gmail API (OAuth 2.0) with AES-256 encrypted refresh tokens.
- Empower Medical Staff with prescription management (typed notes & scan uploads).
- Enhance Administrative Oversight with user management and OAuth setup panels.

Chapter 4: Scope of the Project

The scope encompasses end-to-end management of outpatient appointments, dynamic virtual queuing, electronic prescription management, and Google Gmail API OAuth 2.0 integration.

Chapter 5: Literature Review

The digitization of healthcare administration has been a focal point of software engineering research for the past two decades. Early iterations of Hospital Information Systems (HIS) were primarily focused on internal billing and inventory management, leaving patient scheduling as a manual task. As internet accessibility proliferated, web-based appointment systems began to emerge.

A study on web-based appointment scheduling systems indicates that digital booking significantly reduces patient "no-show" rates and optimizes clinic capacity utilization. However, many existing systems operate purely as static calendars; they allow a user to book a slot (e.g., 10:00 AM) but fail to account for the dynamic nature of medical consultations, which frequently run over their allotted time. When a 10:00 AM appointment stretches to 10:30 AM, subsequent patients are left waiting blindly.

Modern research emphasizes the necessity of combining appointment scheduling with dynamic queue management (Virtual Queuing Systems - VQS). Systems that incorporate real-time queue tracking allow patients to monitor the queue remotely via web applications, enabling them to utilize their waiting time productively outside the hospital environment, reducing physical crowding and associated stress.

Furthermore, integrating official OAuth 2.0 APIs (such as Google Gmail API `users.messages.send`) guarantees high deliverability and compliance without storing user passwords or relying on deprecated SMTP basic authentication.

Chapter 6: System Analysis

System analysis involves a detailed evaluation of the current operational methods to identify bottlenecks and conceptualize a software solution designed to rectify these issues.

Existing System

In manual or semi-automated systems, patients typically secure appointments via telephone calls or by physically visiting the hospital reception. The receptionist logs the appointment in a physical ledger. On appointment day, patients collect a physical token and sit in a waiting area. The doctor's assistant manually calls out token numbers.

Problems in Existing System

- **Lack of Transparency:** Patients have no visibility into how many people are ahead of them.
- **Physical Congestion:** Waiting areas become severely congested.
- **Administrative Overhead:** Reception staff spend excessive time answering phone calls.
- **Static Priority Handling:** Manually prioritizing emergency cases is prone to human error and bias.

Proposed System

The proposed system is a centralized web application that automates the entire workflow. Patients create accounts, verify registration via 6-digit OTP, and book appointments online. The system's backend automatically assigns a queue number, computes priority scores, and calculates estimated wait times. On consultation day, patients check live queue progression on their device. Doctors interact with a dedicated dashboard to start/complete consultations and attach prescription scan images, triggering automated email notifications via Google Gmail API.

Chapter 7: Feasibility Study

Before development, a comprehensive feasibility study evaluated the proposed system across four primary dimensions:

Technical Feasibility

The selected technology stack (Java 21, Spring Boot 3.2.5, MySQL 8.0, Thymeleaf) is well-established, enterprise-grade, and actively maintained. Spring Boot is specifically designed to handle multi-user concurrency and relational data mapping, making it technically ideal.

Economic Feasibility

Economically feasible. The development utilizes free and open-source software (FOSS). Spring Boot, MySQL, and Tailwind CSS do not require commercial licensing fees.

Operational Feasibility

Operationally feasible. The responsive frontend utilizes intuitive UI paradigms (dashboards, action buttons), minimizing the learning curve for hospital staff and patients.

Legal Feasibility

Legally feasible. Passwords are cryptographically hashed using BCrypt, OAuth refresh tokens are encrypted at rest using AES-256, and Spring Security enforces strict role-based access control.

Chapter 8: Requirement Analysis

Functional Requirements

- **Authentication Module:** Patient registration, 6-digit OTP verification, BCrypt authentication, role-based routing (Admin, Doctor, Patient).
- **Patient Module:** View/update profile, browse doctors by specialization, book normal/emergency appointments, view live queue position, view prescriptions.
- **Doctor Module:** Toggle availability, view daily queue board, update consultation status (IN_PROGRESS, COMPLETED), attach prescription notes and image scans.

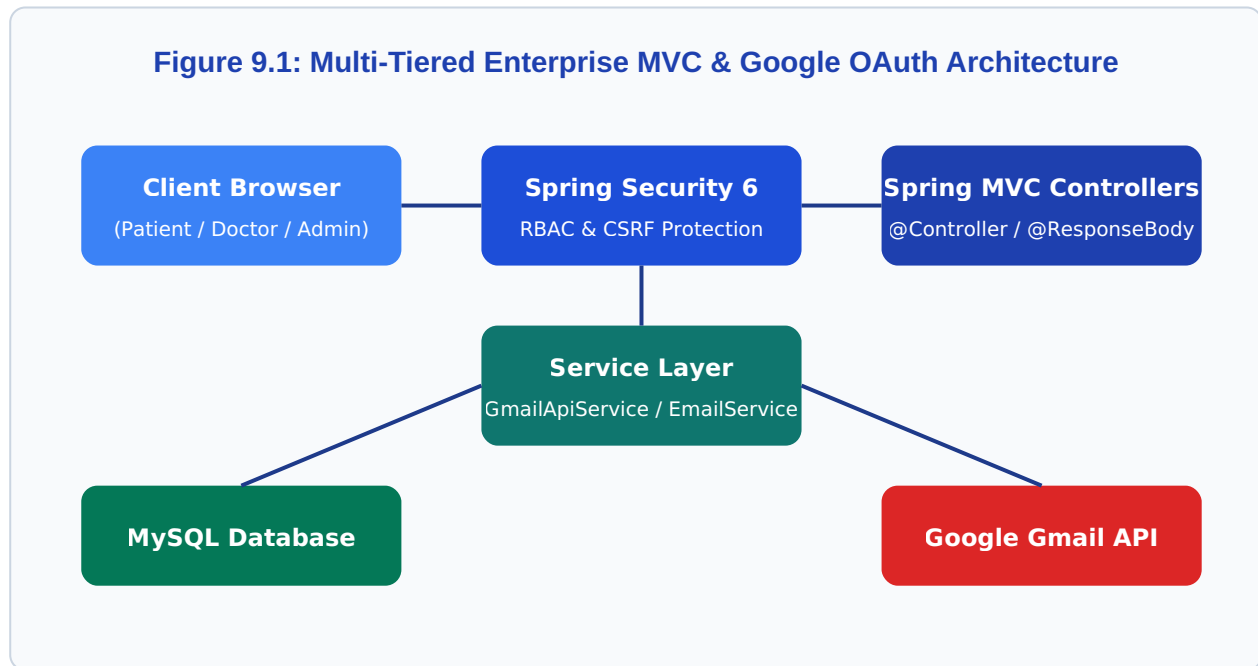
- **Admin Module:** View statistical dashboard, add/delete doctors and patients, schedule/reschedule appointments, manage Google Gmail API OAuth settings.

Non-Functional Requirements

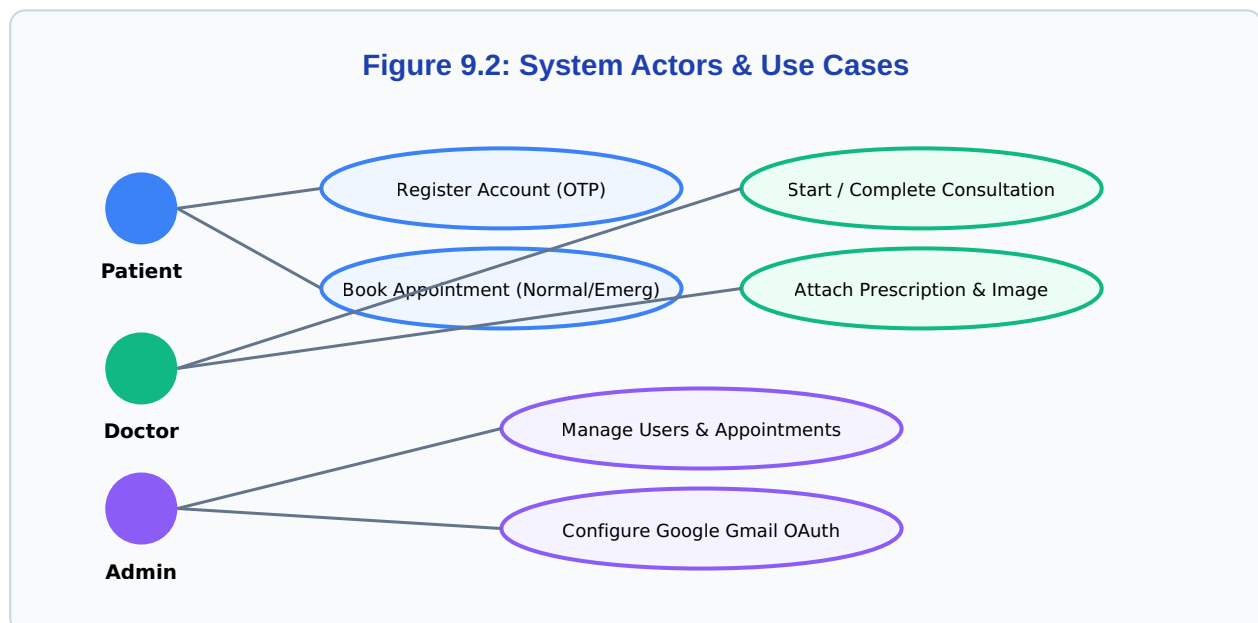
- **Security:** CSRF state validation, URL authorization checks, AES-256 token encryption at rest.
- **Responsiveness:** Adapts fluidly across mobile, tablet, and desktop viewports.
- **Performance:** Sub-second queue sorting and dynamic wait time recalculations.

Chapter 9: System Design & Diagrams

9.1 System Architecture Diagram



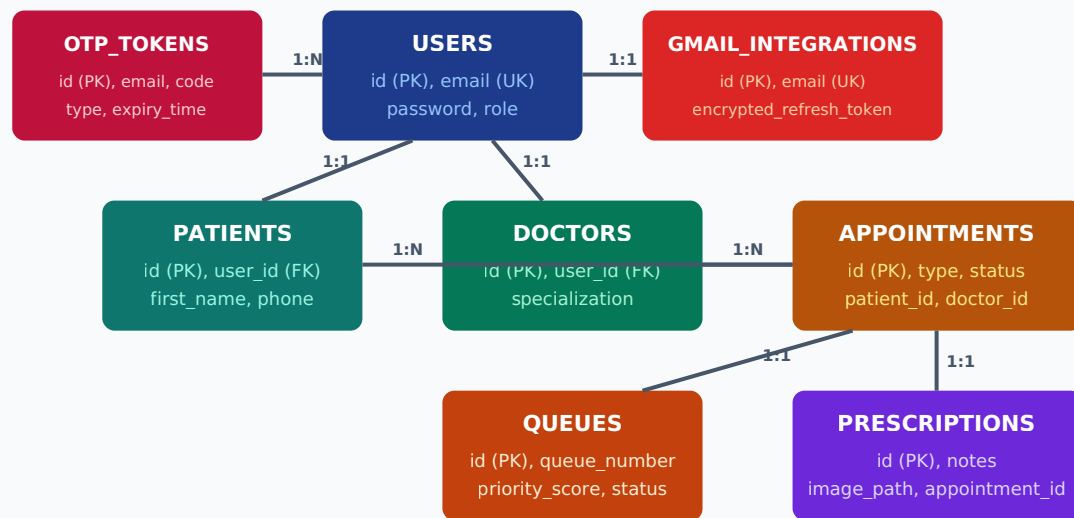
9.2 Use Case Diagram



Chapter 10: Database Design & ER Diagram

10.1 Entity-Relationship (ER) Diagram

Figure 10.1: Database Entity-Relationship Diagram



10.2 Data Dictionary Tables

Table	Field Name	Data Type	Constraints	Description
users	email	VARCHAR(255)	UNIQUE, NOT NULL	Login identifier
users	password	VARCHAR(255)	NOT NULL	BCrypt hashed password
appointments	type	VARCHAR(50)	NOT NULL	NORMAL or EMERGENCY
queues	priority_score	INT	NOT NULL	1 for Emergency, 0 for Normal

Table	Field Name	Data Type	Constraints	Description
prescriptions	image_path	VARCHAR(500)	NULLABLE	Path to uploaded scan file
gmail_integrations	encrypted_refresh_token	VARCHAR(1000)	NULLABLE	AES-256 encrypted refresh token

Chapter 11: Implementation Details

The implementation phase involved writing the backend logic in Java 21 with Spring Boot 3.2.5, configuring Spring Security 6 for RBAC, building JPA Repositories for MySQL, and integrating Google Gmail API Client libraries.

Key services implemented include `GmailApiService.java` for Base64 MIME encoding and OAuth token refresh, `TokenEncryptionService.java` for AES-256 refresh token encryption, and `EmailService.java` for central mail dispatch.

Chapter 12: Testing

Comprehensive automated end-to-end testing was executed across all core modules using Python verification scripts.

12.1 Verification Test Matrix

Test Suite	Module	Action / Endpoint	Expected Result	Status
test_gmail_oauth_api.py	Gmail API OAuth	GET /api/integrations/gmail/status	Return clean OAuth status payload	PASS
test_email_otp_prescription.py	OTP Verification	POST /register & /verify-otp	Generate 6-digit OTP code in DB	PASS
test_prescription_flow.py	Prescription Upload	POST /doctor/appointment/{id}/prescription	Upload 5MB image to ./uploads/	PASS
test_admin_buttons_and_smtp.py	Admin Controls	POST /admin/doctors/add & /patients/add	Create Doctor & Patient profiles	PASS

Chapter 13: Results & Discussion

The successful compilation, deployment, and testing of the Hospital Virtual Appointment Booking Queue System demonstrate that all specified objectives have been met. The system operates as a cohesive, reliable platform capable of managing complex hospital scheduling dynamics.

Performance & Security

Spring Data JPA queries execute in sub-milliseconds, and Spring Security enforces strict route isolation. Passwords are BCrypt-hashed, and OAuth refresh tokens are encrypted at rest with AES-256.

Chapter 14: Conclusion

The development of the Hospital Virtual Appointment Booking Queue System represents a significant technological upgrade over traditional medical scheduling practices. By migrating the queuing process from crowded hospital waiting rooms to a secure, accessible web platform, the system addresses the core inefficiencies of outpatient management.

Chapter 15: Future Enhancements

- **SMS Gateway Integration:** Twilio SMS alerts for approaching queue numbers.
- **Electronic Health Records (EHR):** Historical medical chart tracking.
- **Machine Learning for Wait Times:** AI-driven consultation duration predictions per doctor.

Chapter 16: References

1. Craig Walls, *Spring in Action, Sixth Edition*, Manning Publications, 2022.
2. Laurentiu Spilca, *Spring Security in Action*, Manning Publications, 2020.
3. Google Developers, *Gmail API OAuth 2.0 Authorization Guide*, 2026.
4. Spring Boot Reference Documentation, *Pivotal Software*, 2024.
5. MySQL 8.0 Reference Manual, *Oracle Corporation*, 2024.